

MOATLY

Moat App — Phase 3 Technical Breakdown

Device integration: BLE pairing, combination capture across three tiers, the composite identity code, firmware OTA, and the removal of soft-auth from the codebase.

Prepared for: Engineering lead, Moatly

Document version: 0.1 · Status: Draft for review

Date: April 2026

0. How to read this document

This is the third and final Phase breakdown for the Moat mobile app. *Phase 1* built the app-only MVP; *Phase 2* added Moats; **this document** covers Phase 3, the phase in which the hardware device becomes the authentication path.

Phase 3 is a hardware phase. The heaviest engineering work is on the firmware side (specified in a separate firmware breakdown) and on the industrial design of the Premium and Simple devices. This document covers only the **mobile-app-side** work: the BLE pairing flow, the three combination-capture UIs, the composite identity code composition, the mobile half of firmware OTA, the virtual device software fallback, and the careful removal of soft-auth from the codebase.

The document is shorter than Phase 2. That is correct — Phase 3 on the app side is primarily new UX on top of plumbing that Phase 2 already built (JWT, WebSocket, cycle cascade). The novelty is BLE and the three-tier UX story, not new infrastructure.

0.1 What this document assumes

- The reader has read the Build Brief v1.0, in particular §5.5 (the Device), §5.5.1–4 (tiers, flow, composite code, glow), §6.4 (attestation), §10.4 (Phase 3 scope and exit gate).
- The reader has read the Phase 1 and Phase 2 breakdowns. Modules specified there are referenced freely.
- A parallel firmware breakdown covers the device-side work (Zephyr, BLE GATT profile, PSA Crypto, glow driver, signed DFU). This document points to it where relevant but does not replicate it.

0.2 What this document does not cover

- Firmware. Separate breakdown.
- Industrial design of the enclosures. Owned by the designer and the founder.
- Manufacturing, packaging, journal partner coordination. Owned by the founder + ops.
- Backend changes for device attestation enrolment. Covered by the backend breakdown.

0.3 Document map

§	Contents	Primary reader
1	Decisions required before Week 1 of Phase 3.	Lead, founder
2	Architecture deltas — what's added, what's removed.	All
3	Module catalogue — Phase 3 narrative.	Mobile leads
4	BLE pairing flow, in depth.	Mobile + firmware
5	Combination capture — Premium, Simple, Virtual variants.	Mobile + designer
6	The composite identity code.	Mobile + security

§	Contents	Primary reader
7	Per-module specifications (Phase 3 modules).	Mobile engineers
8	Glow on the app side, firmware OTA, other device concerns.	Mobile + firmware
9	Soft-auth cutover — the operational playbook.	Lead, PM, SRE
10	Cross-cutting, data flows, test strategy.	All
11	Implementation plan, open questions, post-Phase 3 outlook.	Lead, PM

1. Decisions required before Week 1 of Phase 3

Seven decisions. Same format as previous phases: default recommendation, trade-off summary, owner.

1.1 BLE library — react-native-ble-plx, Nordic's Mesh, or custom

Recommendation: react-native-ble-plx, with our own wrapper module that strips anonymous telemetry.

ble-plx is the dominant mature BLE library for React Native, maintained by Polidea and used by Nordic in their reference apps. It handles the platform differences (Core Bluetooth on iOS, android.bluetooth on Android) and exposes a reasonable observable-based API. Any telemetry the library ships with (it has none today, but we verify each upgrade) is stripped via our `scripts/patch-transitive.ts` from Phase 1.

Alternatives considered: Nordic's Android/iOS Mesh SDKs are the native truth but duplicate the effort across platforms and don't speak React Native natively. A fully custom Turbo Module over Core Bluetooth + Android BLE GATT is possible but has no benefit over ble-plx for our use case and adds a full platform-specific maintenance burden.

1.2 RSSI proximity threshold for 'wake' events

Recommendation: -70 dBm for the 'near' threshold, -85 dBm for the 'far' threshold, with 3-second debounce on each.

The brief says 'proximity to the paired phone (Bluetooth RSSI threshold) wakes it' without specifying numbers. -70 dBm is a conservative ~1–2 metre distance on most phones; -85 dBm is roughly 3–5 metres. The device wakes when *any* advertisement from the paired phone is received above the near threshold, and returns to deep sleep when signal drops below the far threshold for 10 consecutive seconds. The separation (-70 vs -85) gives hysteresis that prevents flapping when the user is exactly at the edge.

This is a firmware-side implementation detail; surfaced here because the app emits the advertisement and its advertising interval (recommended 200ms) affects wake latency.

1.3 Virtual device trust level — how loud is the 'reduced security' label

Recommendation: The Virtual Device is labelled 'reduced security' on the Moat home screen banner, at first-use onboarding, and in the account settings. Not as a modal on every session.

The brief is emphatic that Virtual Device is not a fallback-by-default — it is the accessibility path for users who cannot afford hardware. A recurring modal creates a friction tax on exactly the users we want to welcome, which makes it self-defeating. A persistent-but-quiet banner in the app surface is the right pressure: honest, visible, but not punitive.

1.4 First-pair device provisioning — in-store, in-box, or in-app

Recommendation: In-box, with a factory-provisioned attestation key. First pair is a BLE-only dance between device and app; no server round-trip required.

The firmware factory process writes a device-unique attestation keypair into the nRF5340 CryptoCell at manufacture. The app, when it pairs with a fresh device, receives the public half

plus a manufacturer signature that chains to our signing CA. The app enrolls the device with our backend after the user has completed the in-app combination setup. The user never sees 'press the button to enter pairing mode while simultaneously reading a 16-digit code from a sticker.' They open the box, open the app, and the device is offered as a discovered peripheral.

1.5 Combination capture — on-device or server-verified

Recommendation: On-device only. The combination is a local secret that the device uses to unlock its hardware-backed signing key. The server never knows the combination, not even as a hash.

This matches the brief's 'device is the key' principle (§2, principle 5). A combination that the server could verify would be a password with extra steps — and would create an 'I forgot my combination, reset it by email' attack surface we refuse to build. The server verifies device signatures, not user inputs.

1.6 Behavioural fingerprint — what gets hashed, what tolerance

Recommendation: Timing intervals between combination steps, rounded to 50ms buckets. Tolerance: 1-bucket drift per step, up to 3 total. No raw timing data leaves the device.

The composite identity code (brief §5.5.3) has a behavioural component described only as 'a rolling hash of recent combination entry timing and pressure curves.' This breakdown narrows that to: timing only, coarse-grained, with a drift-tolerant match. Pressure curves are a Premium-tier feature and are hashed with the same coarsening. We do not ship raw timing data off the device under any circumstances; the hash is the rolling-fingerprint, and the server sees only the final composite code.

1.7 Summary table

Decision	Default recommendation	Owner
BLE library	react-native-ble-plx + wrapper	Lead + firmware
RSSI proximity thresholds	-70 dBm near, -85 dBm far, 3s debounce	Firmware + mobile
Virtual Device labelling	Persistent banner, not modal	Product
First-pair provisioning	In-box, factory-provisioned keys	Firmware + ops
Combination verification	On-device only	Security
Behavioural fingerprint	Timing buckets, coarse, tolerance-bounded	Security
soft-auth decommission trigger	Hard date, documented in §9	Lead + SRE

2. Architecture deltas from Phase 2

The four-layer architecture is unchanged. Phase 3 adds two new clusters (device + attestation), keeps the Phase 2 clusters intact, and deletes one — the soft-auth module.

2.1 Directory layout — additions and deletions

```
src/
├── crypto/
│   ├── (Phase 1-2 modules, unchanged)
│   └── attestation/
│       ├── composite-code.ts    # (Phase 3) composite identity code
│       ├── behavioural.ts      # rolling timing fingerprint
│       └── firmware-signing.ts  # pinned OTA verification key
├── device/                      # NEW top-level – entire device cluster
│   ├── ble/
│   │   ├── adapter.ts          # ble-plx wrapper; no telemetry
│   │   ├── scanner.ts          # advertise + scan; wake by RSSI
│   │   ├── gatt.ts             # our GATT services + characteristics
│   │   └── types.ts
│   ├── pairing/
│   │   ├── discovery.ts        # find unpaired devices in-box
│   │   ├── enrol.ts           # client side of backend enrolment
│   │   └── re-pair.ts          # from seed phrase → new device
│   ├── combination/
│   │   ├── capture.tsx         # Premium (ring) / Simple (button)
│   │   ├── verify.ts           # on-device unlock
│   │   └── buckets.ts          # 50ms timing bucketisation
│   ├── session/
│   │   ├── handshake.ts        # device challenge + JWT exchange
│   │   └── wakeup.ts           # silent push → device wake → BLE
│   ├── glow/
│   │   └── trigger.ts           # post 'glow-worthy' event to device
│   ├── firmware-update/
│   │   ├── dfu.ts              # signed OTA flow
│   │   ├── manifest.ts         # version + signature verification
│   │   └── rollback.ts         # downgrade protection
│   └── virtual/
│       ├── simulator.ts        # in-app "device" for cost-concession users
│       └── glow-surface.tsx     # on-screen glow equivalent
├── infra/
│   ├── (Phase 1-2 modules, unchanged)
│   └── auth/
│       ├── jwt.ts              # (Phase 2, unchanged)
│       └── soft-auth.ts        # DELETED at Phase 3 T-0
└── ui/
    ├── (Phase 1-2 modules, unchanged)
    └── device/
        ├── onboarding.tsx      # pair-in-box UX
        └── combination-premium.tsx # touch ring
```

```

└─ combination-simple.tsx # press pattern
└─ combination-virtual.tsx # on-screen
└─ not-paired.tsx        # the blocking screen post-soft-auth
└─ settings.tsx          # unpair, replace, update firmware

```

2.2 Module catalogue — Phase 3 additions

Module	Layer	Responsibility	Section
crypto.attestation.composite-code	1	Composite identity code composition.	§6 + §7.1
crypto.attestation.behavioural	1	Timing-bucket rolling fingerprint.	§6 + §7.2
crypto.attestation.firmware-signing	1	Pinned verification key for OTA.	§8.2 + §7.3
device.ble.adapter	2	Wrapper around ble-plx; telemetry-stripped.	§4 + §7.4
device.ble.gatt	2	Our GATT services and characteristic definitions.	§4 + §7.5
device.pairing.discovery	3	Find unpaired devices in the box; user picks one.	§4 + §7.6
device.pairing.enrol	3	Enrol the paired device with the backend.	§4 + §7.7
device.pairing.re-pair	3	Recover a new device from the seed phrase.	§4 + §7.8
device.combination.capture	3	Tier-aware combination capture coordinator.	§5 + §7.9
device.combination.verify	3	On-device unlock (actually a firmware call).	§5 + §7.10
device.session.handshake	3	Device challenge; composite code; JWT exchange.	§6 + §7.11
device.session.wake-up	3	Silent push → BLE connect → handshake.	§8.1 + §7.12
device.glow.trigger	3	Tell firmware to glow when app receives a push.	§8.1 + §7.13
device.firmware-update.dfu	3	Initiate signed DFU from the app.	§8.2 + §7.14
device.virtual.simulator	3	Software-only 'device' for cost-concession users.	§5.3 + §7.15
ui.device.*	4	Pairing UX, three combination UIs, settings.	§5 + §7.16

2.3 Modules deleted at Phase 3 T-0

- `infra/auth/soft-auth.ts` — the soft-auth module itself.
- `ui/(onboarding)/soft-auth.tsx` — the email+seed auth screen.
- Any test fixture referencing soft-auth (moved to the audit-history archive, not the live repo).

\$9 of this document is the playbook for that deletion.

3. Catalogue discussion

The Phase 3 catalogue has five clusters. Each is introduced here before the per-module specifications in §7.

3.1 The BLE cluster

Three modules — adapter, scanner, gatt — that are the mobile side of the conversation with the device. The adapter is the wrapper around ble-plx. The scanner handles advertise/scan for proximity wake. The gatt module owns the characteristic-level protocol: challenge/response, status reads, glow triggers, firmware transfers.

All three are Layer 2 (infrastructure). They do not know what the device is for; they only know how to talk to it.

3.2 The pairing cluster

Three modules — discovery, enrol, re-pair. Discovery shows the user the list of nearby unpaired devices, which should usually be exactly one (the one in their box). Enrol is the backend handshake that binds the device's attestation key to the user's account. Re-pair covers the case where a user lost their first device and is provisioning a replacement using the seed phrase in their journal.

3.3 The combination cluster

Three modules — capture, verify, buckets. Capture is the only cluster that changes shape per device tier: the Premium ring collects a gesture sequence, the Simple button collects a press pattern, the Virtual device collects an on-screen equivalent. Verify is the call across BLE to unlock the hardware-backed key. Buckets converts raw timing into 50ms-aligned values that go into the behavioural fingerprint.

3.4 The session cluster

Two modules — handshake and wakeup. Handshake owns the five-step auth flow from the brief (§5.5.2): proximity wake → combination → device signs challenge → composite identity code → JWT. Wakeup coordinates with the existing silent-push path from Phase 1 — when the push arrives, wakeup kicks the BLE connection, handshake runs, and the Moat is ready to open by the time the user looks at the device.

3.5 The firmware-update cluster

Three modules — dfu, manifest, rollback. DFU is the actual over-BLE bytes-pushing flow (we use Nordic's Secure DFU profile). Manifest verifies the OTA image against our pinned signing key before flashing. Rollback is the downgrade-protection: a monotonic version counter enforced by firmware, so that a stolen or fraudulent older-signed image cannot be pushed.

3.6 The virtual-device cluster

One module (plus a UI component). The virtual device simulates every part of the device protocol *inside the phone*. It holds no hardware-backed key — the 'key' is stored in the phone's secure element (iOS Secure Enclave / Android Keystore), which is weaker than the dedicated

nRF5340 CryptoCell but still hardware-backed on modern phones. The glow surface is a full-screen on-device animation styled to resemble the physical device's halo.

The virtual device is a first-class path, not a fallback.

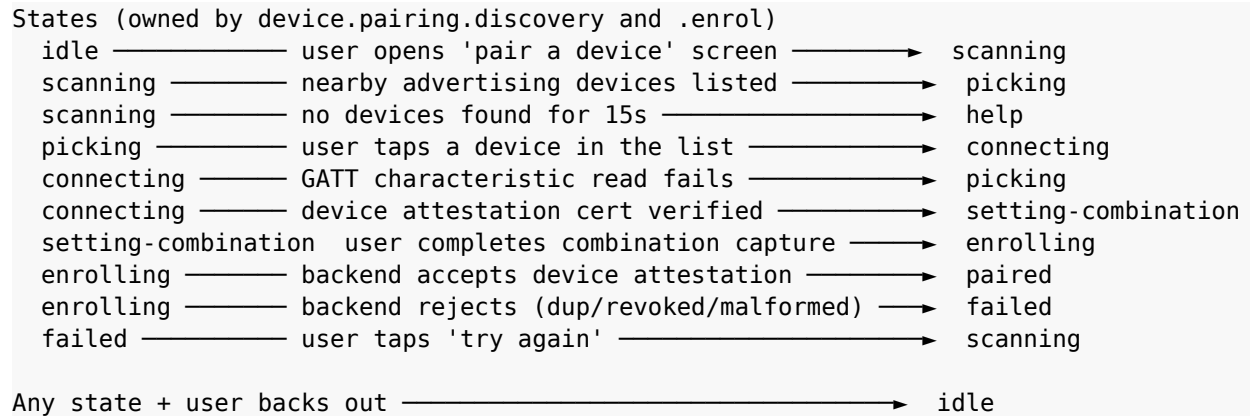
Cost-concession users deserve a path that is not visibly lesser. The virtual device's UX should be quiet, warm, and obviously-Moatly — not a 'you're missing out' framing. The 'reduced security' label is there because it is true, not because we want virtual users to feel excluded.

4. BLE pairing — in depth

Pairing is the first moment the user touches the device. It is the moment where the device's attestation key is enrolled with the backend, binding that physical object to that account forever (or until explicit revocation). It is also the moment a new user forms their first impression of whether the hardware 'just works' or 'is going to be a project.'

For both reasons, the pairing flow gets disproportionate engineering attention.

4.1 The pairing state machine



4.2 Discovery — what the user sees

In the common case: user opens the box, taps 'pair a device,' and waits 5–10 seconds while the app scans. A single card appears with 'Device · <pseudonym pair>,' where the pseudonym pair is derived client-side from the device's advertising bytes so the user has a way of knowing 'this is my device' when there are multiple boxes in a shipping facility or a coffee shop.

In the uncommon case: multiple devices are advertising. The list shows them with their pseudonyms and signal strength as a silhouette of dots. The user taps the one they intend. A mis-tap is recoverable — we prompt 'is this yours?' after connecting, and the combination-capture step won't work for a device that isn't actually in the user's possession.

4.3 The attestation step

On first connect we read three GATT characteristics:

- `attestation.device_pubkey` — the factory-provisioned public key half.
- `attestation.factory_cert` — a certificate signed by our factory CA binding the pubkey to a device serial.
- `attestation.challenge_response` — the device signs a challenge from the app with its private key.

The app verifies (1) the factory cert chains to our pinned CA, (2) the challenge signature validates under the advertised pubkey, (3) the device serial is not in a revocation list the app ships with. If all three pass, the device is trusted as *coming from us*. Enrolment with the backend follows.

4.4 Enrolment with the backend

The app POSTs to `/v1/accounts` (for new accounts) or `/v1/devices/enrol` (for a replacement) with:

```
{
  "account_id":      "...",           // if replacing, else null
  "covenant_hash":   "...",           // from Phase 1 flow
  "device_attestation": {
    "hw_pubkey":      "base64...",
    "factory_cert":   "base64...",
    "challenge_nonce": "base64...",   // the challenge we issued
    "signature":      "base64..."   // device's signature over it
  },
  "client_version":  "ios-1.0.0"
}
```

The backend verifies the same three things independently and, if satisfied, records the device as paired. The response is a JWT and an `account_id`.

Crucially: the user's combination is NOT part of this request. It never is. The combination unlocks the device's signing key locally; the backend only ever sees signatures.

4.5 Re-pairing — the lost-device recovery path

When a user buys a new device because they lost or broke their old one, the pairing flow works with one addition: after the attestation step succeeds on the new device, the app prompts for the 24-word seed phrase from the journal. The seed phrase derives the master key (Phase 1 §4.2), which unlocks the SQLCipher DB and identity key material on the new phone.

The backend enrolment call includes an `'account_id'` taken from that DB, so the server knows to associate the new device's attestation key with the existing account. The old device's attestation key is not revoked yet — it is tombstoned with a 14-day grace so the user who finds their lost device tucked into the sofa cushions the next week does not have to start over. After 14 days the old key is revoked and the old device is a paperweight.

4.6 Background and iOS quirks

- iOS Core Bluetooth preserves state restoration across app terminations for connected peripherals but not for scan operations. We request the central manager with `CBCentralManagerOptionRestoreIdentifierKey` so the connected-device state survives a restart but we re-scan on every foreground.
- Android background BLE is handled by `ble-plx`'s background mode; we declare the `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` permissions (Android 12+) and live with Doze restrictions.
- Users see the OS-native pairing prompt exactly once, at first pair. We do NOT use a 16-digit pairing code; we use the factory cert + challenge as the security mechanism, and the OS prompt is 'allow Bluetooth access,' not 'enter a code.'

5. Combination capture — three variants

The combination is the user's memorised secret that unlocks the device's signing key. The firmware knows how to verify a combination; the app's job is only to capture the user's input and hand it to the device via BLE. What differs across tiers is the capture UX — everything below the capture (bucketisation, behavioural fingerprint, key unlock) is identical.

5.1 Premium — the capacitive touch ring

The user enters a sequence of taps on the ring's capacitive surface. During onboarding the user chooses a pattern of 5 to 7 gestures, each of which is one of: tap-at-12, tap-at-3, tap-at-6, tap-at-9, swipe-CW, swipe-CCW, long-press. The firmware knows the eight gesture types and maps each to a small integer. The app shows a muted visualisation of the pattern as it is entered (a trail of dots around a virtual ring on screen).

On every subsequent sign-in the user re-enters the pattern. The firmware verifies and unlocks the signing key if correct; the app sees only 'unlocked / not unlocked' plus, on success, the signed challenge response.

5.1.1 Fingerprint capture on Premium

Premium tier sends back per-gesture timing and a coarsened pressure envelope (the capacitive sensor can estimate contact area, which we bucket into 4 levels). Phones the firmware-coarsened version back to the app for the behavioural-fingerprint step.

5.2 Simple — the single-button press pattern

One hardware button. The combination is a press pattern: a sequence of short and long presses and inter-press pauses. The pauses carry information — a combination like short-short-LONG-pause-short encodes timing the firmware can verify. Length 8–14 inputs (press or pause), depending on what the user chooses at onboarding.

Simple tier sends only press timings for the fingerprint — there is no pressure data.

Simple is not 'worse.' It is different.

The Simple tier's cryptographic guarantees are identical to Premium (same nRF5340 CryptoCell, same PSA Crypto key, same attestation chain). The only thing missing is the pressure dimension of the behavioural fingerprint, which provides a small additional layer of same-person assurance. Simple users do not lose access to anything, ever. This framing matters in the onboarding copy.

5.3 Virtual — the on-screen device

The virtual device renders an on-screen ring that looks like a simplified Premium device. The user taps around the virtual ring with the same 8-gesture vocabulary as Premium. On a touchscreen, pressure is estimated from *3D Touch* (where available) or touch area (Android), but most phones don't meaningfully vary contact pressure, so virtual tier effectively uses timing-only fingerprint.

The 'signing key' for virtual devices lives in the phone's secure element: iOS Secure Enclave (iPhone 5s and later) or Android Keystore (hardware-backed on most devices from 2018 onward). The attestation chain is different — the factory cert is self-generated on the phone at

first launch and submitted to the backend during enrolment. The backend records `virtual_device=true` on the row so that anti-Sybil rate limits can be adjusted if needed.

5.3.1 Virtual glow

Per Phase 1 §4.12, push payloads carry only { `wakeup: true` }. When a virtual device receives one, the virtual-glow-surface module renders a full-bleed amber pulse on the phone screen for 8 seconds (matching the Premium glow event visually), then dismisses. If the phone is backgrounded, the silent push has already wakened the app enough to run the glow surface briefly. iOS's silent-push throttling means this is best-effort.

5.4 Onboarding the combination — an explicit ritual

Choosing a combination is a moment of consequence. The onboarding copy is explicit: this is the secret that unlocks your Moatly. Nobody can reset it for you. Do not reuse a passcode from another context. Do not choose something obvious. Write a hint in the journal, but not the combination itself.

Technically: the user enters their chosen combination three times during onboarding. The three entries must agree; if they do not, the user starts over. On acceptance, the firmware stores the combination as the PSA Crypto unlock credential for the signing key.

5.5 Failed-entry lockdown

Firmware enforces: after 5 consecutive failed entries, the device enters a 60-second lockdown. After 10, 10 minutes. After 15, 24 hours. After 20, the device wipes its signing key and requires re-pairing from seed phrase.

The app surfaces these states plainly. No shame copy; just a clock and 'try again when you can.'

6. The composite identity code

The composite identity code is the mobile-computed token that the server uses to answer *this is the same person as last time?* — without ever being able to answer *who is this person?*. The brief specifies the three components (§5.5.3); this section specifies how the mobile app composes them.

6.1 The three components

Component	Source	Mobile app's role
Hardware attestation	Firmware signs the server-issued challenge with the device's PSA Crypto key.	Forwards the challenge over BLE; forwards the signature up to the backend.
Behavioural fingerprint	Rolling hash of recent combination timing (and, on Premium, pressure) buckets.	Maintains the rolling fingerprint state across sessions in SQLCipher; computes the hash per-session.
Server nonce	Issued by the backend at the start of each session challenge.	Passes through unchanged into the composite hash input.

6.2 The composition function

```
composite_identity_code(session) =
  HKDF-SHA256(
    ikm   = device_signature || server_nonce,
    salt  = 'moatly.composite.v1',
    info  = 'session.' || session_id,
    okm_length = 32
  ) XOR
  HMAC-SHA256(
    key   = behavioural_fingerprint_rolling_hash,
    msg   = server_nonce
 )[:32]
```

Inputs:

device_signature — 64 bytes, Ed25519 over server_nonce.
 server_nonce — 32 bytes, single-use.
 behavioural_fp_hash — 32 bytes, updated on every successful entry.

Output:

32 bytes, single-use per session. Sent to the backend.

The XOR of the two halves binds both the hardware signature and the behavioural fingerprint into the code, so that a legitimate device with a wrong fingerprint (e.g. someone who stole the device and guessed the combination on the 19th try) produces a *different* code than the genuine owner would. The server does not know the fingerprint, so it cannot detect this directly — but anomaly detection on the server (e.g. 'this code doesn't match any historical code's nonce-invariant component') can flag it.

6.3 The rolling fingerprint

After every successful combination entry, the fingerprint updates:

```
behavioural_fp_hash_new =
  HMAC-SHA256(
    key = behavioural_fp_hash_old,
    msg = bucketed_entry_timings_for_this_session
  )
```

```
bucketed_entry_timings = for each gesture/press in the combination:
  round_down_to_50ms(time_since_previous_input)
```

The rolling fingerprint is stored in SQLCipher under the user's master-key-derived sub-key (Phase 1 §4.2). It is never transmitted. The only thing the backend sees is the final `composite_identity_code`, which is a 32-byte HMAC output — computationally-infeasible-to-invert back to the fingerprint.

6.4 Drift tolerance — preventing the 'bad coffee day' lockout

People don't enter their combination at identical timing every time. The fingerprint's role is to say *roughly the same person*, not *an exact timing match*. Tolerance is handled on the server side: the backend stores the last N (we recommend N=10) composite codes the user's account successfully produced, and a new incoming code is accepted if it matches any of them within the bucket's drift window, OR if the user has produced, within the last 90 days, enough *pattern-drift tolerance* to cover a gradual change. This is a server-side concern — the mobile app composes the code honestly and lets the server decide.

6.5 What the code does not do

- It does not identify the user across accounts. Different account → different rolling fingerprint → different codes.
- It does not fingerprint the device across installs. Uninstall and re-install from scratch → new key, new fingerprint.
- It does not contain the user's name, email, or any PII.
- It does not persist beyond the session. The server stores 'seen before?' metadata, not the code itself.

7. Per-module specifications

Same template as v0.1 and Phase 2: Responsibility, Public interface, Data, Dependencies, Notes, Risks, Acceptance criteria. Modules are ordered by layer (crypto → infra → domain → UI).

7.1 crypto.attestation.composite-code

Responsibility

Compose the composite identity code as specified in §6.2. Does not fetch inputs; accepts them as explicit arguments so the module is deterministic and testable.

Public interface

```
export function composeCompositeCode(inputs: {
  deviceSignature: Uint8Array; // 64 bytes
  serverNonce:    Uint8Array; // 32 bytes
  behaviouralFpHash: Uint8Array; // 32 bytes
  sessionId:      string;
}): Uint8Array; // 32 bytes
```

Data

None. Pure.

Dependencies

- @noble/hashes for HKDF and HMAC.

Acceptance criteria

- Given identical inputs, output is identical (determinism). 10,000-iter property test.
- Any single-bit change in any input changes at least 128 output bits on average (avalanche).
- Test vectors match a reference implementation (to be produced by the security engineer for external audit).

7.2 crypto.attestation.behavioural

Responsibility

Maintain the rolling behavioural fingerprint. Update it on each successful combination entry. Load and persist it via SQLCipher.

Public interface

```
export async function loadFingerprint(): Promise<Uint8Array>;

export async function updateFingerprint(
  bucketedTimings: Uint8Array
): Promise<Uint8Array>; // returns the new fingerprint

export async function initialiseFingerprint(
  masterKey: MasterKeyHandle
```

```
): Promise<void>; // at first pair; derived from seed
```

Data

```
CREATE TABLE behavioural_fingerprint (
  id          INTEGER PRIMARY KEY CHECK (id = 1),
  fingerprint BLOB NOT NULL,      -- 32 bytes
  updated_at  INTEGER NOT NULL
);
```

Dependencies

- crypto.keyring for initial derivation.
- infra.db for persistence.

Acceptance criteria

- The fingerprint value never leaves the module except into composeCompositeCode.
- Initialisation with the same master key produces a deterministic starting fingerprint.
- A test that attempts to read the fingerprint via any public API outside this module fails the module-boundary static analysis.

7.3 crypto.attestation.firmware-signing

Responsibility

Hold the pinned OTA verification public key. Expose one function: 'does this manifest's signature verify under the pinned key?' Keys are compiled into the app binary; rotating them requires a mobile-app release.

Public interface

```
export function firmwareSigningPublicKey(): Uint8Array;

export function verifyFirmwareManifest(
  manifest: Uint8Array,
  signature: Uint8Array
): { valid: boolean; reason?: string };
```

Data

Pinned key is a constant in the source. Rotating requires a release with the new key appended to a short ordered list — during a rotation window both old and new keys are accepted, then the old is removed in the next release.

Acceptance criteria

- A modified manifest with a valid signature from the old key continues to verify until the rotation-window release.
- A manifest signed by anything other than the pinned key(s) fails verification.

7.4 device.ble.adapter

Responsibility

Wrap react-native-ble-plx into a typed, telemetry-stripped, application-appropriate surface. Expose only the operations we actually use. Every other module in device.ble.* talks to ble-plx through this adapter, not directly.

Public interface

```
export async function scanForDevices(
  opts: { serviceUuid: string; timeoutMs: number }
): Promise<DiscoveredDevice[]>;

export async function connectToDevice(
  deviceId: string
): Promise<ConnectedDevice>;

export async function readCharacteristic(
  device: ConnectedDevice,
  serviceUuid: string,
  characteristicUuid: string
): Promise<Uint8Array>;

export async function writeCharacteristic(
  device: ConnectedDevice,
  serviceUuid: string,
  characteristicUuid: string,
  value: Uint8Array
): Promise<void>;

export async function subscribeToCharacteristic(
  device: ConnectedDevice,
  serviceUuid: string,
  characteristicUuid: string,
  handler: (value: Uint8Array) => void
): Promise<Unsubscribe>;

export async function disconnect(device: ConnectedDevice): Promise<void>;
```

Notes

- No 'scanForever' or 'autoReconnect' options. Callers decide when to reconnect.
- No event emitters that expose raw ble-plx types. Every value returned is a typed object from device/ble/types.ts.

Acceptance criteria

- An ESLint rule rejects any import of react-native-ble-plx from outside this module.
- All timeouts are caller-specified; no hardcoded network timeouts that would mask bugs.

7.5 device.ble.gatt

Responsibility

Define the GATT service UUIDs, characteristic UUIDs, and their framing. Provide typed higher-level operations (read-attestation, write-challenge, read-signature, write-glow-event, subscribe-to-status).

Public interface

```
export const MOATLY_SERVICE_UUID      = '5f6d6174-6c79-4d4f-0001-000000000000';
export const MOATLY_ATTESTATION_UUID  = '5f6d6174-6c79-4d4f-0001-000000000010';
export const MOATLY_CHALLENGE_UUID    = '5f6d6174-6c79-4d4f-0001-000000000011';
export const MOATLY_SIGNATURE_UUID     = '5f6d6174-6c79-4d4f-0001-000000000012';
export const MOATLY_GLOW_UUID          = '5f6d6174-6c79-4d4f-0001-000000000020';
export const MOATLY_STATUS_UUID        = '5f6d6174-6c79-4d4f-0001-000000000030';
export const MOATLY_DFU_CONTROL_UUID   = '5f6d6174-6c79-4d4f-0001-000000000040';
export const MOATLY_DFU_DATA_UUID      = '5f6d6174-6c79-4d4f-0001-000000000041';

export async function readAttestation(d: ConnectedDevice): Promise<Attestation>;
export async function issueChallenge(d: ConnectedDevice, nonce: Uint8Array):
Promise<Uint8Array>;
export async function sendGlowEvent(d: ConnectedDevice, event: GlowEvent):
Promise<void>;
export async function subscribeToStatus(d: ConnectedDevice, h: (s: DeviceStatus) =>
void): Promise<Unsubscribe>;
```

Acceptance criteria

- The service and characteristic UUIDs are identical to the firmware's. Mismatch is a build-time fail via a shared constants file.
- All framing is covered by round-trip tests against a firmware-in-a-loop emulator on CI.

7.6 device.pairing.discovery

Responsibility

Run the 'pair a device' state machine described in §4.1. Scan for advertising Moatly devices; present the list; let the user pick.

Public interface

```
export interface DiscoveredDevice {
  deviceId: string;
  displayPseudonym: string; // two words from advertising bytes
  rssi: number;
}

export function usePairingDiscovery(): {
  state: 'idle' | 'scanning' | 'picking' | 'connecting' | 'paired' | 'failed';
  devices: DiscoveredDevice[];
  startScan(): void;
  pick(deviceId: string): Promise<void>;
  reset(): void;
};
```

Acceptance criteria

- Scanning times out after 15 seconds with a 'no devices found' state that offers help (in-app link to the troubleshooting guide).

7.7 device.pairing.enrol

Responsibility

After the user has picked a device and set a combination, enrol the device with the backend. Binds the attestation key to the account.

Public interface

```
export async function enrolDevice(args: {
  deviceId: string;
  attestation: Attestation;
  accountId?: string;    // null for new account, set for re-pair
}): Promise<{ jwt: string; accountId: string }>;
```

Acceptance criteria

- Enrolment is idempotent on the server side: re-enrolling the same device for the same account succeeds without creating a duplicate row.
- A revoked or tombstoned attestation key is rejected with a clear error the UI can explain.

7.8 device.pairing.re-pair

Responsibility

The replacement-device flow. Validate the seed phrase, load the existing account, enrol the new device, tombstone (not yet revoke) the old device's attestation key.

Public interface

```
export async function repairFromSeed(args: {
  seed: string[];        // 24 BIP-39 words
  newDeviceId: string;
  newAttestation: Attestation;
}): Promise<{ jwt: string; accountId: string; oldDeviceTombstonedUntil: number }>;
```

Acceptance criteria

- A correct seed phrase grants access; an incorrect one fails with 'seed phrase does not match any account' after a 2-second intentional delay (anti-brute-force).
- After re-pair, the old device continues to work for 14 days, then stops.

7.9 device.combination.capture

Responsibility

Tier-aware coordinator. Looks up the connected device's tier from its attestation metadata, renders the correct capture UI, collects input, converts to bucketed timings via device.combination.buckets.

Public interface

```
export type CaptureResult = {
  combination: Uint8Array; // the input sequence
  bucketedTimings: Uint8Array; // for fingerprint
};

export async function captureCombination(
  tier: 'premium' | 'simple' | 'virtual',
  purpose: 'onboarding' | 'verify'
): Promise<CaptureResult | { cancelled: true }>;
```

Acceptance criteria

- Premium captures 5–7 gestures; Simple captures 8–14 presses; Virtual captures the same gestures as Premium via on-screen ring.
- The capture UI can be cancelled at any point by back-swipe or explicit cancel button.

7.10 device.combination.verify

Responsibility

Send the captured combination to the device over BLE; receive 'unlocked / not unlocked' in return. This does not expose the combination to the app's process memory for longer than a single function call.

Public interface

```
export async function verifyCombinationOnDevice(
  device: ConnectedDevice,
  captureResult: CaptureResult
): Promise<{ unlocked: boolean; remainingAttempts?: number; lockoutUntil?: number }>;
```

Acceptance criteria

- The combination buffer is overwritten with zeros immediately after BLE transmission (verified via a test hook).
- A locked-out device returns a lockoutUntil timestamp which the UI displays.

7.11 device.session.handshake

Responsibility

Run the 5-step auth flow from §5.5.2 of the brief. Used on every foreground-to-moat transition (unless a valid JWT is already live from recent activity).

Public interface

```
export async function runHandshake(
  device: ConnectedDevice
): Promise<{ jwt: string; expiresAt: number }>;

// Internal steps, in order:
// 1. POST /v1/auth/challenge → server_nonce
// 2. writeCharacteristic(CHALLENGE_UUID, server_nonce)
```

```
// 3. captureCombination('verify') + verifyCombinationOnDevice
// 4. readCharacteristic(SIGNATURE_UUID) → device_signature
// 5. composeCompositeCode + POST /v1/auth/verify → jwt
```

Acceptance criteria

- A handshake that fails at any step returns a typed error that the caller can branch on; no raw string messages.
- Total handshake latency, once the user has completed their combination, is <2 seconds p95.

7.12 device.session.wakeup

Responsibility

When a silent push arrives (new message in the user's Moat), pre-warm the BLE connection so that, when the user picks up the device, the handshake is ready. This is how we target glow-to-read <5s on Android, <15s on iOS (brief §14.1).

Public interface

```
export function onWakeupPush(handler: () => Promise<void>): Unsubscribe;
```

```
// Internal wakeup logic:
// 1. Silent push arrives → JS runs for up to 30s on Android, ~10s iOS.
// 2. Ensure BLE is scanning/connected.
// 3. Trigger a glow event on the device without waiting for handshake.
// 4. Pre-fetch any new messages from the WebSocket if it's alive.
// 5. Exit; user's next interaction finds the socket warm.
```

Acceptance criteria

- In a field test, device-glow-to-app-open-and-messages-visible is measured at p95 <5s on mid-range Android and <15s on iPhone SE. Tests on 2018-era phones; newer phones should meet these bounds.

7.13 device.glow.trigger

Responsibility

Send a glow event to the connected device via BLE. Encodes the five event types from brief §5.5.4.

Public interface

```
export type GlowEvent =
  | { kind: 'major_statement' }
  | { kind: 'cycle_begin' }
  | { kind: 'ring_completed' }
  | { kind: 'low_battery' }
  | { kind: 'stop' };

export async function triggerGlow(
  device: ConnectedDevice,
  event: GlowEvent
```

```
): Promise<void>;
```

Notes

- No 'five new messages' event exists. The brief is explicit that only Major Statements trigger glow.
- 'low_battery' is triggered by the device itself reporting status <15%, not by the app. The app may forward the event for analytics (shape-only counter), but the firmware is authoritative.

7.14 device.firmware-update.dfu

Responsibility

Perform a signed, over-BLE firmware update using Nordic's Secure DFU profile. Verify the manifest signature locally before initiating transfer. Abort cleanly if the device reports any integrity error mid-transfer.

Public interface

```
export async function checkForUpdate(
  device: ConnectedDevice
): Promise<{ available: boolean; currentVersion: string; latestVersion?: string }>;

export async function downloadManifest(
  version: string
): Promise<{ manifest: Uint8Array; signature: Uint8Array; imageUrl: string }>;

export async function performUpdate(
  device: ConnectedDevice,
  version: string,
  onProgress?: (pct: number) => void
): Promise<void>;
```

Notes

- Updates are opt-in, not silent. The settings screen shows 'firmware update available; takes about 90 seconds' and the user taps to start. This matches the brief's 'update via the app' phrasing without auto-updating hardware without consent.
- Downgrade protection: the firmware enforces a monotonic version counter. A manifest for an older version is rejected by the firmware itself, not just by the app. We still validate in the app to avoid wasting bandwidth.

Acceptance criteria

- An update with a valid signature installs successfully in <2 minutes over BLE on a modern phone.
- An update with an invalid signature is rejected by the app before any bytes are transferred.
- An update with a version older than the device's current version is rejected by the firmware.

7.15 device.virtual.simulator

Responsibility

Implement the entire device protocol inside the app, using the phone's secure element for key storage. Surfaces the same API as a real device to the rest of the app so that the handshake, capture, glow, and firmware-update modules do not need to branch on tier.

Public interface

```
export async function createVirtualDevice(): Promise<ConnectedDevice>;

// Internal: generates an Ed25519 keypair in the OS secure element,
// self-signs a factory-equivalent cert (flagged virtual_device=true on
// enrolment), and registers an in-process GATT-equivalent interface
// that the rest of device.* talks to identically to a real device.
```

Notes

- The virtual device has no over-BLE surface — it runs in the same process. The `ConnectedDevice` object returned is a façade that maps the BLE operations to local function calls.
- Virtual devices support firmware-update-equivalent: the app itself is the firmware, so 'updating the device' means updating the app. The DFU flow short-circuits with 'your virtual device updates with the app.'

Acceptance criteria

- The rest of device.* cannot tell a virtual device from a physical one at the interface boundary — the same test suite passes for both (parameterised on tier).
- Virtual device key material is stored in the OS secure element, not in SQLCipher or plain memory.

7.16 ui.device.*

Responsibility

Six screens: the pair-in-box onboarding flow, three combination UIs (Premium, Simple, Virtual), the 'you are not paired' blocking screen for post-soft-auth users, and the device settings screen (unpair, replace, firmware update).

Components

Component	What it does
PairInBoxOnboarding	Scan list, attestation indicator, combination setup. One screen that advances through states internally rather than a hard stack.
CombinationPremium	Virtual ring UI that mirrors the physical device's ring. The on-screen ring lights at the position the user is currently touching. Muted dot trail shows the pattern-so-far.
CombinationSimple	A single large on-screen button that shows visual feedback on press. A timeline beneath it shows the press pattern as it is built.
CombinationVirtual	Same UI as CombinationPremium, on-screen as both capture and identity.

Component	What it does
	No separate 'device' artefact.
NotPairedBlocker	A full-screen explanation: 'The old sign-in method has ended. To continue, pair a device or use the Virtual Device.' Two large buttons. This is shown to soft-auth users at Phase 3 T-0.
DeviceSettings	Device name, battery, last-paired, firmware version, 'check for update', 'unpair this device', 'replace with a new device via seed phrase.'

Acceptance criteria

- The three combination UIs are visually distinct but share a common design language. A Maestro test opens each and confirms the correct widget renders based on the paired device's tier.

8. Glow, firmware OTA, and other device concerns

8.1 The glow path — end to end

Glow-to-read latency is a named product metric. Here is how the full path works on the app side:

```
T=0      Another Moat member posts a Major Statement.
T+~1s    The Moat service sends a silent push to every other member's
         phone: { wakeup: true }.
T+~2s    The phone wakes the app's background JS.
T+~2s    device.session.wakeup fires:
         (a) ensures BLE is connected to the user's device
         (b) calls device.glow.trigger({ kind: 'major_statement' })
         (c) pre-fetches new messages over WebSocket
T+~3s    The device halo begins its slow amber pulse.
T+~Xs    The user notices the glow and picks up the phone/device.
T+X+<2s  User opens app → device.session.handshake runs (cached
         connection, pre-fetched messages). The Moat is visible.
```

Total "glow to read": $T + X + <2s$.

Since X is user-latency, the engineering target is the part we can control: glow appears within ~3s of the post, and the Moat is visible within ~2s of the app open.

We cannot make X smaller. We can make the parts around X invisible.

The engineering win is not 'the glow appears 500ms faster.' It is 'when the user's gaze falls on the lit device and then their phone, the Moat is already open by the time their eyes focus.' Pre-fetching at silent-push time is the mechanism. A Moat that greys in after the user opens the app feels slow regardless of raw latency.

8.2 Firmware OTA — the signing side

The firmware signing key lives offline on an air-gapped machine held by the security engineer and the founder (two-of-two). Manifests are signed at release time. The public half is pinned in the app (§7.3). A compromise of the signing key requires:

- An emergency app release that replaces the pinned key.
- A public incident note per §12.4 of the Build Brief.
- Revocation of all outstanding signed manifests from whatever version was compromised.

There is no server-side key rotation mechanism. This is deliberate — a rotating server-side key would be a single point of failure that could be compelled by a state-level adversary to sign a malicious manifest.

8.3 Device revocation

Lost device: the user runs 'replace device' (§7.8). The old device is tombstoned for 14 days, then revoked.

Stolen device: same flow, but the user can choose 'revoke immediately' instead of 'tombstone for 14 days.' This requires re-authentication via seed phrase on the new device.

Fraud / abuse: the backend can revoke a device's attestation key in response to a confirmed abuse incident (see brief §5.10). The revoked device gets its key zeroed on the next successful handshake attempt (firmware-enforced, because the app cannot be trusted to self-brick).

8.4 Battery and power

- The app displays the device's battery percentage in settings; the device itself shows two quick red blinks when below 15% (brief §5.5.4).
- The app never drives the device's glow for 'low battery' — that is firmware-only. If the firmware glow driver is misbehaving, the app has no override.
- Expected battery life at typical use (per firmware target): 30+ days between charges.

9. Soft-auth cutover — the operational playbook

The Phase 2 breakdown (§7.3) described the cutover in principle. This section is the operational playbook — who does what, when, in what order. It is written to be run by the engineering lead, PM, and SRE as a checklist.

9.1 Pre-cutover preparation (T-45 days)

- **Audit soft-auth usage.** Query backend metrics: how many accounts are soft-auth'd? How many are active in the last 30 days? Share with founder and lead.
- **Confirm Phase 3 firmware and app releases are on track for the cutover date.** If they are not, postpone the cutover — do not ship the soft-auth-removal release early.
- **Prepare the NotPairedBlocker UX (§7.16).** Copy reviewed by founder, legal, accessibility checked.

9.2 T-30 days — the announcement

- **Email to all soft-auth'd users via the email relay:** 'Your device pairing starts in 30 days. The old email-plus-seed sign-in ends at that time. If you have a physical device, pair it now to avoid friction. If you will use the Virtual Device, no action needed.'
- **Status page update.** Publish a brief post on status.moatly.[tld] announcing the cutover date and rationale.
- **In-app banner.** Soft-auth'd users see a persistent top banner 'pair a device by <date>' with a 'pair now' button.

9.3 T-14 days — partial refuse

- **Backend release.** The `/v1/auth/soft-auth-verify` endpoint begins returning a 'deprecated; new JWTs will not be issued after <T-0 date>' warning header. Existing JWTs continue to work.
- **Reminder email.** Same distribution as T-30, refreshed text.

9.4 T-0 — the cutover release

- **App release ships.** Contains: all Phase 3 device modules, NotPairedBlocker, no soft-auth module. PR that deleted `infra/auth/soft-auth.ts` is merged and released.
- **Backend release.** `/v1/auth/soft-auth-*` endpoints begin returning 410 Gone.
- **Final email.** 'Today is the day. Pair a device from inside the app.'

9.5 T+7 days — endpoint removal

- **Backend release.** `/v1/auth/soft-auth-*` endpoints are deleted from the router. The 410 Gone path is removed.
- **Code archive.** Git branch 'archive/soft-auth' is created from the last commit containing soft-auth, for audit trail. Main branch no longer references soft-auth anywhere.

9.6 Contingency

If firmware slips, hold the cutover.

The one condition that must be true on T-0 is: the Virtual Device path is fully working for every soft-auth'd user who wants to continue. If anything in the Phase 3 app release is broken on T-0, we delay the cutover by a week. Shipping the NotPairedBlocker without a working pairing flow is the one mistake that would genuinely lock out real users.

10. Cross-cutting, data flows, test strategy

10.1 Permissions and privacy prompts

Phase 3 introduces two new OS-level prompts: Bluetooth and (on iOS 13+) 'nearby devices.' The copy for each prompt is controlled by the app's Info.plist / AndroidManifest. We match the rest of Moatly's editorial voice: plain, short, honest.

- iOS NSBluetoothAlwaysUsageDescription: 'Moatly uses Bluetooth to talk to your paired device. No data about you or your device is sent anywhere else.'
- Android BLUETOOTH_SCAN / BLUETOOTH_CONNECT: Same text in the runtime prompt.
- No analytics on whether the user accepted, declined, or dismissed.

10.2 State management additions

- useDeviceStore — connection status, battery, tier, paired-at.
- usePairingFlowStore — current step of the pairing wizard.
- useCombinationCaptureStore — ephemeral, wiped when capture completes.

10.3 Data flows

10.3.1 First-time device setup (new user)

1. User completes Phase 1 onboarding (covenant, seed phrase).
2. New in Phase 3: 'pair a device' step instead of skipping.
3. User taps 'I have a device' (or 'use Virtual Device').
4. Discovery scan → picking → connecting → attestation verified.
5. Combination capture (Premium / Simple / Virtual, tier-aware).
6. User enters combination 3x, 3x agreement enforced.
7. Device stores combination as PSA Crypto unlock credential.
8. app.device.pairing.enrol → POST /v1/accounts with attestation.
9. Backend returns JWT. User lands on Moat home.

10.3.2 Daily sign-in

1. App foregrounded (or silent push wakes it).
2. Existing JWT still valid → no handshake needed. Go to Moat home.
3. Existing JWT expired → device.session.handshake:
 - a. Connect to device over BLE (already cached if warm).
 - b. Capture combination (user enters it).
 - c. Device signs challenge → composite identity code.
 - d. POST /v1/auth/verify → new JWT.
4. Go to Moat home.

10.3.3 Lost-device recovery

1. User buys a replacement device.
2. Opens app → 'pair a device' → discovers new device.
3. Attestation verified → prompt: 'Is this a replacement?'
4. User taps yes → re-pair flow:
 - a. Prompts for 24-word seed phrase.
 - b. Derives master key → loads existing account_id.
 - c. Sets a new combination on the new device.

- d. Enrols new device under the same account.
 - e. Old device tombstoned for 14 days.
5. App returns to normal operation with the existing account.

10.4 Test strategy — Phase 3 additions

Test	What it proves
PAIR-1 — Happy path pair	A simulated device (Rust firmware-in-a-loop) is discovered, attested, enrolled, and usable for a handshake, in under 60s.
PAIR-2 — Wrong factory cert	A device whose cert does not chain to our CA is rejected at attestation. No enrolment request is sent.
PAIR-3 — Two devices in range	Multiple advertising devices show separate cards. Picking one enrolls only that one.
COMBO-1 — Correct combination	Correct input → device signs → handshake completes.
COMBO-2 — Wrong combination	Wrong input → device refuses → UI shows 'try again' with attempts remaining.
COMBO-3 — Lockout escalation	5 wrong entries → 60s lockout. 10 → 10min. 15 → 24h. 20 → wipe and re-pair required.
VIRTUAL-1 — Parity	Every PAIR-* and COMBO-* test passes unchanged when the device is a virtual device.
DFU-1 — Happy OTA	Signed manifest → full DFU transfer → device reboots into new firmware. Version counter incremented.
DFU-2 — Bad signature	Tampered manifest → app rejects before any bytes transferred. User sees 'update failed.'
DFU-3 — Downgrade attempt	Older-version manifest → firmware rejects. App surfaces the rejection.
LAT-1 — Glow to read	Silent push → glow → app opens → Moat visible, p95 <5s Android, <15s iOS. Measured on reference phones (Pixel 7, iPhone SE 3).
SOFT-AUTH-1 — Post-cutover	After T-0, a build with soft-auth still referenced fails CI. After T-7, calling /v1/auth/soft-auth-* returns 410.

11. Implementation plan, open questions, post-Phase 3

11.1 Weekly plan

Phase 3 budgets 10–12 weeks. This plan walks a 12-week path with two mobile engineers plus one firmware engineer whose weeks are synchronised.

W	Mobile engineer A (device + attestation)	Mobile engineer B (UI + virtual device)
1	ble.adapter + ble.gatt; firmware-in-a-loop harness.	ui.device.PairInBoxOnboarding scaffold.
2	pairing.discovery + .enrol; end-to-end against dev FW.	CombinationPremium component (visual ring).
3	attestation.composite-code + .behavioural + tests.	CombinationSimple component (press timeline).
4	session.handshake; 5-step flow against dev FW.	CombinationVirtual + virtual-device simulator v0.
5	session.wakeup; silent push integration.	Virtual device simulator v1 (full parity).
6	glow.trigger; end-to-end glow-to-read measurement.	DeviceSettings screen (battery, FW version, unpair).
7	firmware-update.dfu + manifest verification.	Firmware update UX (progress, cancel, rollback).
8	re-pair flow (seed → new device).	NotPairedBlocker; copy polish.
9	Cross-tier parity tests (real + simulated).	Accessibility pass across all device UX.
10	Test suite: PAIR-*, COMBO-*, DFU-*, LAT-*.	100-person cohort onboarding materials.
11	100-person cohort dogfood bug-burn.	100-person cohort dogfood bug-burn.
12	Soft-auth deletion PR; Phase 3 release candidate.	Phase 3 release candidate.

11.2 Dependencies on firmware

- Week 1: firmware-in-a-loop harness runs our GATT profile.
- Week 2: dev firmware accepts challenges and returns signatures.
- Week 4: dev firmware implements combination verify + lockout.
- Week 6: dev firmware supports glow events.
- Week 7: dev firmware supports Secure DFU with our signing scheme.
- Week 10: production-candidate firmware ready for cohort.

11.3 Dependencies on backend

- Week 2: /v1/accounts accepts device attestation payloads.
- Week 4: /v1/auth/challenge and /v1/auth/verify support composite-code auth.

- Week 8: `/v1/devices/revoke` and `/v1/devices/tombstone` endpoints.
- Week 12 (T-0 release + 1 week): `/v1/auth/soft-auth-*` removed.

11.4 Open questions

Question	Owner	Needed by	Default if no answer
Confirm GATT service UUIDs — base pattern vs fully random.	Firmware + mobile	Before Week 1	Use a Moatly-specific base pattern (see §7.5); easier to debug.
Confirm exact RSSI near/far thresholds.	Firmware + field test	Before Week 5	-70 / -85 dBm with 3s debounce (§1.2).
Combination vocabulary: confirm 8 gestures for Premium, 2 for Simple.	Designer + firmware	Before Week 2	Matches §5.1, §5.2 of this document.
Pressure levels on Premium — 4 or 8 buckets.	Firmware + security	Before Week 3	4 buckets. More is more fingerprint identifying info.
Revocation grace: 14 days, or shorter?	Founder + security	Before Week 8	14 days (§4.5).
Firmware update UX: progress bar, or narrative?	Designer + product	Before Week 7	Both — a progress bar with quiet copy ('updating...').
Virtual-device 'reduced security' banner wording.	Product + founder	Before Week 5	'Virtual device — reduced security. Works the same day-to-day.'
100-person cohort mix — how many Premium / Simple / Virtual?	Founder + ops	Before Week 11	60 Premium, 30 Simple, 10 Virtual per brief §10.4.

11.5 What comes after Phase 3 — the post-launch outlook

Phase 4 in the Build Brief (§10.5) is public beta, external audit, and open-source repo activation. On the mobile side, Phase 4 is mostly not new code — it is:

- Remediating audit findings on the codebase.
- Setting up the public repo as the primary development remote.
- Writing the app's contribution guide, threat-model docs, and architecture decision records in public-facing form.
- Supporting TestFlight / Play open beta operations.

Post-launch, the mobile cadence slows deliberately. Feature velocity is not the goal. Depth of practice is.

11.6 Closing

At the end of Phase 3, Moatly is the product the Build Brief described: a privacy-first practice platform where the hardware device is the key, the Moat content is cryptographically destroyed every seven days, and no part of our infrastructure can answer the question 'what did this user say?'

The remaining work — Phase 4 and beyond — is operational and editorial rather than architectural. Get this shipped, get it audited, publish the repo, hold the line on the principles, and let the platform become what it is for.

— *End of Phase 3 Technical Breakdown* —

Moatly · Moat App Phase 3 technical breakdown · April 2026 · v0.1